Clarity - Complete Pipeline & Technology Guide for Judges

How to Explain the End-to-End Architecture, AI Pipeline & Engineering Decisions | Team AiX Global

1. The 30-Second Elevator Pitch (Technical)

"Clarity is an evidence-first review-to-resolution system built on Next.js 16, PostgreSQL, Drizzle ORM, Jev, and OpenAI. Unlike generic AI dashboards that summarize reviews into vague hallucinations or auto-write empty apologies, Clarity enforces a strict provenance pipeline: we save the raw review first, screen it with Jev (with automatic OpenAI fallback), extract atomic observations anchored to exact character-level sentence spans so the AI can never invent a quote, cluster them into branch-specific operational issues, track staff repair reports with photo evidence, and only unlock AI reply drafting AFTER an owner verifies the repair-while blocking any reply that claims 'everything is fixed' if another complaint in the same review is still open."

2. The 6-Stage End-to-End Pipeline (Step-by-Step)

Stage 1: Durable, Idempotent Intake (Before AI Touches Anything)

* What happens: When a review enters POST /api/action (type='intake'), Zod validates the business, property/branch, rating (1-5), timestamp, and text (5-3000 chars). The raw originalText is immediately written to PostgreSQL as 'pending'.
* Why it matters to judges: If an AI API times out or crashes, the customer evidence is never lost. Furthermore, we enforce a unique constraint on (business_id, source, source_review_id). Replaying the same feed is idempotent, but if someone tries to alter the text or rating under the same ID, the backend throws a Feed Conflict error to protect evidence integrity.

Stage 2: Fast AI Screening with Automatic Fallback (Jev -> OpenAI)

* What happens: POST /api/action (type='analyze') calls our screening layer (screenWithFallback). In live mode, it first hits the Jev API (jev-latest) with an 8-second timeout and 2 retries.
* The Probability Rule: Jev returns a 'noul' score between 0 and 1. If score >= 0.6, the review has specific actionable feedback. If score <= 0.4, it is marked 'generic' (e.g., 'Great place!') so it stays searchable in the database without cluttering the repair board.
* Resilience Fallback: If Jev returns an uncertain score (between 0.4 and 0.6) or experiences a network error, our backend automatically falls back to OpenAI (gpt-5-mini) using the exact same criteria, logging the fallback timing in analysis_runs.stage_timings.

Stage 3: Zero-Hallucination Quote Extraction (Intl.Segmenter + Zod Enum)

* What happens: A single review often mixes multiple points (e.g., broken AC, a bathroom insect, and polite front desk staff). OpenAI splits the review into independent observations.
* How we prevent AI quote hallucination: We do NOT let the LLM type out the quote freely. Before calling OpenAI, our backend uses JavaScript's native Intl.Segmenter to split the review into numbered sentence spans (s1, s2, s3...). We dynamically compile a Zod schema where evidenceSpanId is a strict enum of those IDs. OpenAI must pick the span ID, and our verifyEvidence() guard verifies the exact substring offsets in the original text.

Stage 4: Smart Issue Matching & Anti-Inflation Triage Engine

* What happens: Each observation is matched to an existing Theme (Issue) at the same property/branch and of the same feedback kind (operational_complaint, suggestion, or specific_praise).
* Cost & Latency Optimization: Before calling OpenAI, candidateThemes() filters themes by lexical word overlap and boosts themes with active tasks. If an exact title match exists or zero candidates overlap, we skip the LLM call entirely. Unmatched complaints in live mode automatically spawn a new suggested issue.
* Distinct-Review Support & Priority: Support is calculated via COUNT(DISTINCT review_id) so one angry review repeating the same complaint 4 times only counts as 1 review. Priority (Urgent, High, Medium, Low) combines owner severity (1-3), total distinct reviews, recent reviews (180 days), and post-resolution recurrence.

Stage 5: Operational Task Workflow, Advisory Locks & Photo Evidence

* What happens: Owners confirm an issue and propose a repair task. We use a PostgreSQL transaction-level advisory lock (pg_advisory_xact_lock) so double-clicking or concurrent requests cannot create duplicate tasks.
* Simulated Orbyte Handoff: When the owner assigns a branch employee and due date and approves the task, the backend snapshots all supporting review quotes and writes a simulated Orbyte handoff payload locally.
* Staff Reporting & Optimistic Concurrency: Staff submit a completion report (min 10 chars) plus up to 5 base64-validated photos (JPEG/PNG/WebP, <= 2MB each). Every submission increments report_version. When the owner verifies or rejects the report, the SQL UPDATE checks report_version to prevent stale overwrites.
* Automatic Recurrence Detection: If a new review about the same issue arrives after a task was resolved (review.createdAt > task.resolvedAt), the backend automatically flips the issue status to 'recurrence', bumps priority to High, and unlocks a linked follow-up task.

Stage 6: Truthful Reply Drafting & Partial-Resolution Guardrails

* What happens: Replies are locked until a linked task is verified as 'resolved'. When drafting, unresolvedReviewComplaints() checks every OTHER observation in that same review.
* The Partial-Reply Guardrail: In our demo review, the guest complained about BOTH the Room 226 AC and a bathroom insect. If only the AC task is resolved, the backend passes the unresolved insect complaint into the OpenAI prompt so the draft only claims the AC was fixed and notes the insect concern is still under review.
* Regex Universal-Claim Blocker: Even if an owner manually edits the reply, assertReplyApproval() scans the text with regex rules and blocks approval if the text claims 'all issues are resolved' or 'everything is fixed' while another complaint remains open. Batch approval (up to 300 replies) is strictly blocked for partial-warning or Severity 3 issues.

3. Why We Chose Each Technology (Judge Justification)

* Next.js 16 (App Router, Node.js Server Runtime) + React 19: Keeps all database queries, Jev keys, and OpenAI keys strictly server-side while serving the interactive Ledger workspace from a single unified codebase deployable on Replit.
* PostgreSQL 16 + Drizzle ORM: Relational integrity is essential because one review -> many observations -> many themes -> tasks -> 1 reply per review. Drizzle gives us type-safe SQL, ACID transactions, conditional version updates, and PostgreSQL advisory locks.
* Zod (Runtime Validation + Structured Outputs): Acts as a two-way firewall-validating untrusted HTTP inputs on /api/action and enforcing strict JSON schema outputs (zodTextFormat) from OpenAI's Responses API.

* Jev (TypeSafe API) + OpenAI (gpt-5-mini): Dual-provider architecture. Jev provides fast, specialized binary probability screening ('noul' score), while OpenAI handles fallback screening, multi-observation span extraction, semantic theme clustering, and context-aware reply drafting.
* Dual AI_MODE (fixture vs. live): Allows deterministic, zero-latency judge testing and CI test runs in 'fixture' mode, while supporting real end-to-end Jev + OpenAI execution in 'live' mode.

4. Fast Answers to Tough Judge Questions (Q&A Cheat Sheet)

* Q: 'How do you stop the AI from hallucinating quotes or making up fake promises in replies?' -> A: Two ways. For extraction, the AI isn't allowed to generate quote strings; it can only select sentence span IDs (s1, s2) segmented by Intl.Segmenter, which we substring-verify against the raw review. For replies, drafting is blocked until a manager verifies a staff report, and our regex blocker rejects any reply claiming all issues are fixed when secondary complaints in the same review remain open.
* Q: 'What happens if the AI API goes down or returns garbage?' -> A: First, the review is saved to PostgreSQL before AI runs, so data is never lost. Second, if Jev fails or is uncertain (0.4-0.6), OpenAI automatically takes over screening. Third, if extraction fails, the review is marked 'failed' (retryable) or can be manually classified by the owner. Fourth, if reply drafting fails, the system generates a clearly labeled deterministic template_fallback draft.
* Q: 'How do you handle multi-tenancy and data isolation between branches?' -> A: Every API call runs assertActionScope() before executing domain logic, verifying that the review, observation, theme, task, and reply all belong to the selected businessId, and themes/employees are strictly partitioned by branchId (e.g., Central vs. Garden property).
* Q: 'Is this connected to live Google Reviews or Orbyte right now?' -> A: By design, no external scraping or live public posting happens in the prototype. We built clean adapter boundaries (localHandoff and simulatePublish) so judges can safely test the entire operational lifecycle without posting synthetic data to public review platforms.